claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4\agent-a88c0884018f3a9c3.jsonl`
- SHA-256: `b9dfbad52af2bc360ff4f298edb721d93a6b9ee0f2e87d8a9179a64d2d9ba45f`
- Source modified: `2026-06-18T08:16:19+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_87c49086-be4`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T08:05:23.416Z)

```
You implement ONE behavior-preserving refactor and open a PR for OWNER REVIEW (do NOT merge).
You are in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS
machine.
The Python env has pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore
any
CLAUDE.md note saying it can't).

SETUP FIRST, exactly:
  git fetch origin
  git checkout -B <BRANCH> origin/master      # base MUST be origin/master, NOT the current HEAD
then verify: `git log --oneline -1` is origin/master's HEAD and `git branch --show-current` ==
<BRANCH>.

RULES (pure refactor ? quality must NOT regress, behavior must NOT change):
- Behavior-PRESERVING moves/extractions ONLY. Do NOT change logic, branch conditions, log
  strings/levels, return types, ordering, thresholds, or constants. It is cut-paste into
helpers.
- Edit ONLY the files named in the task. Stage EXPLICIT paths (`git add <file>`), NEVER -A/./-u.
- Match the surrounding code style and naming.

GATE (binding ? run all four; capture pass/skip counts + coverage %):
  python run_all_tests.py
  python -m ruff check .
  python -m mypy backend training
  python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
ONLY IF ALL FOUR ARE GREEN:
  git add <explicit files>
  git commit -m "<conventional commit msg>"   # final line: Co-Authored-By: Claude Opus 4.8
<noreply@anthropic.com>
  git push -u origin <BRANCH>
  gh pr create --base master --head <BRANCH> --title "<title>" --body "<concise body: what/why,
behavior-preserving argument, gate results>"   # NO merge
IF ANY GATE IS RED: do NOT push or open a PR. Fix if it's a trivial slip you introduced;
otherwise
report status='gate-failed' with the failing output and your diff so far. Never weaken a test or
lower the coverage floor to go green.

Return the structured object with: slug, branch, status ('pr-opened'|'gate-failed'|'error'),
pr_url (""+reason if none), diffstat, suite (e.g. '851 passed, 7 skipped'), ruff, mypy,
coverage,
behavior_note (why it's a pure move), caveat.

=== YOUR TASK: main-uploads-router (branch refactor/main-uploads-apirouter) ===
FILES: backend/main.py + NEW backend/api/uploads.py (create backend/api/__init__.py if the
```

package doesn't exist). Extract the self-contained chunked-upload island from main.py ? the in-process
registry + helpers + 4 endpoints: _uploads, _uploads_lock, _upload_cfg, _abort_upload, the UploadInitRequest / UploadCompleteRequest pydantic models, and
upload_init/upload_part/upload_complete/
upload_abort ? into backend/api/uploads.py exposing `router = APIRouter()` carrying the 4 routes.
In main.py replace the inline defs with `from backend.api import uploads as uploads_api` and `app.include_router(uploads_api.router)`. Keep identical route paths, status codes, body-size limits,
sequential-part logic, and abort cleanup. THE ONE CRITICAL THING: uploads.py must read the SAME module-level global_config object that main.py sets at startup ? import/resolve it the SAME way main.py
does (e.g. via the existing accessor or `from backend.main import ...`/a shared config module); if it
reads a different/stale config it's a real regression. test_upload_download.py monkeypatches the upload
registry and reads global_config off backend.main ? RETARGET those monkeypatch targets to the new
module (test-only edit, behavior-preserving for production). This is MEDIUM risk ? be meticulous about
the config-global seam; if you cannot make it read the live startup config cleanly, STOP and report
status='gate-failed' rather than shipping a regression.
TESTS: tests/test_upload_download.py (init/part/complete/abort, body-limit, sequential-part rejection,
abort cleanup) + full suite.

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-7 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T08:05:25.929Z)

I'll start by setting up the worktree branch from origin/master, then explore the relevant files.

3. user (2026-06-18T08:05:30.914Z)

branch 'refactor/main-uploads-apirouter' set up to track 'origin/master'.
Switched to a new branch 'refactor/main-uploads-apirouter'
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
---BRANCH---
refactor/main-uploads-apirouter

4. user (2026-06-18T08:05:31.275Z)

a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)

5. assistant (2026-06-18T08:05:33.938Z)

Branch is set up correctly on origin/master HEAD (a010667). Now let me explore the upload island in main.py and the test file.

6. user (2026-06-18T08:05:36.690Z)

871 backend/main.py
static
no api dir

7. user (2026-06-18T08:05:37.750Z)

```
65:global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
121:    return global_config
144:    val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
148:    default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
190:        segmenter = segmenter_cls(db_instance, global_config)
229:            db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
355:    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
409:_uploads: Dict[str, Dict[str, Any]] = {}
410:_uploads_lock = threading.Lock()
413:class UploadInitRequest(BaseModel):
418:class UploadCompleteRequest(BaseModel):
422:def _upload_cfg():
423:    sec = getattr(global_config, "security", None)
431:def _abort_upload(upload_id: str) -> None:
432:    with _uploads_lock:
433:        st = _uploads.pop(upload_id, None)
442:def upload_init(req: UploadInitRequest):
444:    upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
460:    with _uploads_lock:
461:        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes":
0}
467:async def upload_part(upload_id: str, index: int, request: Request):
469:    _, max_bytes, part_bytes, _ = _upload_cfg()
470:    with _uploads_lock:
471:        st = _uploads.get(upload_id)
491:        _abort_upload(upload_id)
494:    with _uploads_lock:
501:def upload_complete(upload_id: str, req: UploadCompleteRequest):
503:    upload_dir, _, _, _ = _upload_cfg()
504:    with _uploads_lock:
505:        st = _uploads.get(upload_id)
516:    with _uploads_lock:
517:        _uploads.pop(upload_id, None)
525:def upload_abort(upload_id: str):
527:    with _uploads_lock:
528:        known = upload_id in _uploads
531:    _abort_upload(upload_id)
548:    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
596:    stitching = getattr(global_config, "stitching", None) or StitchingConfig()
619:    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
751:        # global_config isn't initialized this early; load the typed config so the trim
772:    global global_config, db_instance
773:    global_config = load_app_config()  # returns typed AppConfig
778:    global_config.output.output_dir = args.output_dir
779:    os.makedirs(global_config.output.output_dir, exist_ok=True)
781:    db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
802:        val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
809:        seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
834:            segmenter = segmenter_cls(db_instance, global_config)
850:                db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
```

8. assistant (2026-06-18T08:05:41.039Z)

Let me read the full upload island and the surrounding context (head of main.py for imports, and the global_config setup).

9. user (2026-06-18T08:05:42.293Z)

```
      import os
      import sys
      import argparse
      import logging
      import threading
      import traceback
      import uuid
      from concurrent.futures import ThreadPoolExecutor
      import uvicorn
      from dotenv import load_dotenv

      load_dotenv()

      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
      logging.basicConfig(
          level=logging.INFO,
          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
          datefmt='%H:%M:%S'
      )
      logger = logging.getLogger(__name__)
      from fastapi import FastAPI, HTTPException, Request
      from fastapi.middleware.cors import CORSMiddleware
      from pydantic import BaseModel
      from typing import List, Dict, Any, Optional

      from fastapi.staticfiles import StaticFiles
      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
      from backend.pipeline.validators.base import registry
      from backend.infrastructure.database import Database
      from backend.pipeline.segmenters.base import segmenter_registry
      from backend.pipeline.stitcher.compiler import VideoCompiler
      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
      from backend.results import Failure  # standardized error/result contract
      from backend.reporting import emit_indexer_report

      app = FastAPI(title="Sports Highlight Indexer API")

      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
      app.add_middleware(
          CORSMiddleware,
          allow_origins=["*"],
          allow_credentials=False,
          allow_methods=["*"],
          allow_headers=["*"],
      )

      # Serves static frontend files directly (now under api/ per approved structure)
      os.makedirs("backend/api/static", exist_ok=True)
      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

      @app.get("/", response_class=HTMLResponse)
      def serve_index():
          index_path = "backend/api/static/index.html"
          if os.path.exists(index_path):
              with open(index_path, "r", encoding="utf-8") as f:
                  return f.read()
          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
```

```
backend/api/static/</h3>"
62
63      # Global variables initialized at startup
64      db_instance: Optional[Database] = None
65      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
66
67      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70      # already parallelize their AI handoff internally via their own pools.
71      _job_executor: Optional[ThreadPoolExecutor] = None
72
73      def _get_executor() -> ThreadPoolExecutor:
74          """Lazy module-global executor; tests monkeypatch this for inline execution."""
75          global _job_executor
76          if _job_executor is None:
77              _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78          return _job_executor
79
80      # Legacy thin wrapper for any remaining direct calls during transition.
81      # New code should import load_app_config / AppConfig from backend.config directly.
82      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83          cfg = load_app_config(config_path)
84          return cfg.model_dump(mode="json")
85
86      # --- API Models ---
87      class ProcessRequest(BaseModel):
88          video_path: str
89          player_pool: Optional[List[str]] = None
90          max_frames: Optional[int] = None
91          segmenter: Optional[str] = None
92
93      class QueryRequest(BaseModel):
94          video_id: str
95          server: Optional[str] = None
96          receiver: Optional[str] = None
97          duration_min: Optional[float] = None
98          event_type: Optional[str] = None
99
100     class CompileRequest(BaseModel):
101         video_id: str
102         segment_ids: List[int]
103         output_filename: str
104
105     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
108         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
109         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
110         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
111         """
112         if segments:
113             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
114         else:
115             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
116
117
118     # --- API Endpoints ---
119     @app.get("/api/config")
120     def get_config():
121         return global_config
122
```

```
123     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
124         """The full hash ? validate ? segment ? report pipeline for one video.
125 
126         This is the former synchronous /api/process body, unchanged in behavior, now run on
127         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
128         the sync endpoint used to return (UI compatibility); the per-video observability
129         report is still emitted in `finally:` and grafted as response["reports"].
130 
```

10. user (2026-06-18T08:05:42.880Z)

```
400 
401     # --- Chunked upload (B2, PR-2) ----------------------------------------------------
402     # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403     # proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
404     # under security.upload_dir, then submitted to /api/process by the returned path.
405     # Upload state is in-process by design (this is the single-box alpha): a server restart
406     # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407     # arrives with PR-3 identity scoping.
408 
409     _uploads: Dict[str, Dict[str, Any]] = {}
410     _uploads_lock = threading.Lock()
411 
412 
413     class UploadInitRequest(BaseModel):
414         filename: str
415         total_size_bytes: Optional[int] = None
416 
417 
418     class UploadCompleteRequest(BaseModel):
419         total_parts: int
420 
421 
422     def _upload_cfg():
423         sec = getattr(global_config, "security", None)
424         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
425         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
426         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
427         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
428         return upload_dir, max_bytes, part_bytes, exts
429 
430 
431     def _abort_upload(upload_id: str) -> None:
432         with _uploads_lock:
433             st = _uploads.pop(upload_id, None)
434         if st:
435             try:
436                 os.remove(st["tmp_path"])
437             except OSError:
438                 pass
439 
440 
441     @app.post("/api/upload/init")
442     def upload_init(req: UploadInitRequest):
443         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
444         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
445         try:
446             name = safe_output_filename(req.filename)
447         except ValueError as e:
448             raise HTTPException(status_code=400, detail=str(e)) from e
449         ext = os.path.splitext(name)[1].lower().lstrip(".")
450         if ext not in exts:
451             raise HTTPException(status_code=400,
452                                 detail=f"Extension '.{ext}' not allowed. Allowed:
```

```
{sorted(exts)}")
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(status_code=413,
                                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
        os.makedirs(upload_dir, exist_ok=True)
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
        return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
                "next_part": f"/api/upload/{upload_id}/part/0"}


    @app.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(status_code=409,
                                detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")

        received = 0
        overflow = None
        with open(st["tmp_path"], "ab") as fh:
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
                    break
                fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


    @app.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
```

```
509             raise HTTPException(status_code=409,
510                                 detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511         final_path = os.path.join(upload_dir, st["filename"])
512         if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
513             stem, ext = os.path.splitext(st["filename"])
514             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515         os.replace(st["tmp_path"], final_path)
516         with _uploads_lock:
517             _uploads.pop(upload_id, None)
518         video_id = compute_video_id(final_path)
519         return {"path": os.path.abspath(final_path), "video_id": video_id,
520                 "size_bytes": os.path.getsize(final_path),
521                 "next": "POST /api/process with this path"}
522
523
524     @app.delete("/api/upload/{upload_id}")
525     def upload_abort(upload_id: str):
526         """Abort an in-flight upload and delete its partial file."""
527         with _uploads_lock:
528             known = upload_id in _uploads
529         if not known:
530             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531         _abort_upload(upload_id)
532         return {"status": "aborted", "upload_id": upload_id}
533
534
535     # --- Highlight download (B3, PR-2) ------------------------------------------------
536
537     @app.get("/api/highlights/{video_id}/{filename}")
538     def download_highlight(video_id: str, filename: str):
539         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
```

11. assistant (2026-06-18T08:05:47.341Z)

Now I have the full upload island (lines 401-532). The critical seam is `global_config`, read in `_upload_cfg()` at line 423. Since `global_config` is set at startup in `main.py`'s `global_config = load_app_config()`, the new module must read the *live* value. The cleanest behavior-preserving approach: in `uploads.py`, do `from backend import main as main_module` and reference `main_module.global_config` (so it always reads the current module-level global, not a stale copy).

Let me check how the test monkeypatches things to confirm the exact targets.

12. user (2026-06-18T08:05:48.223Z)

```
1       """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3       Contract under test:
4       - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5       - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort
(413)
6       - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
7       - DELETE /api/upload/{id} ? abort removes partial state
8       - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
9       """
10      import os
11
12      import pytest
13
14      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
15      from fastapi.testclient import TestClient
16
```

```
    import backend.main as m
    from backend.infrastructure.database import Database
    from backend.config.models import AppConfig
    from backend.utils.hashing import compute_video_id


    @pytest.fixture
    def env(tmp_path, monkeypatch):
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()
        cfg.output.output_dir = str(tmp_path / "out")
        cfg.security.upload_dir = str(tmp_path / "up")
        os.makedirs(cfg.output.output_dir, exist_ok=True)
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_uploads", {})  # isolate upload state per test
        client = TestClient(m.app)
        return client, db, tmp_path, cfg


    def _init(client, filename="match.mp4", total=None):
        body = {"filename": filename}
        if total is not None:
            body["total_size_bytes"] = total
        return client.post("/api/upload/init", json=body)


    # --- init ------------------------------------------------------------------------

    def test_init_rejects_bad_extension(env):
        client = env[0]
        r = _init(client, "malware.exe")
        assert r.status_code == 400
        assert "not allowed" in r.json()["detail"]


    def test_init_rejects_traversal_filename(env):
        client = env[0]
        assert _init(client, "../evil.mp4").status_code == 400
        assert _init(client, "a/b.mp4").status_code == 400


    def test_init_rejects_oversize_declared_total(env):
        client, db, tmp_path, cfg = env
        cfg.security.max_upload_mb = 1
        r = _init(client, total=2 * 1024 * 1024)
        assert r.status_code == 413


    # --- full lifecycle --------------------------------------------------------------

    def test_roundtrip_assembles_parts_and_matches_md5(env):
        client, db, tmp_path, cfg = env
        up = _init(client).json()
        parts = [b"a" * 1000, b"b" * 2000, b"c" * 500]
        for i, blob in enumerate(parts):
            r = client.put(f"/api/upload/{up['upload_id']}/part/{i}", content=blob)
            assert r.status_code == 200
            assert r.json()["received_parts"] == i + 1
        done = client.post(f"/api/upload/{up['upload_id']}/complete", json={"total_parts":
3})
        assert done.status_code == 200
        body = done.json()
        assert body["size_bytes"] == 3500
        with open(body["path"], "rb") as fh:
```

```
            assert fh.read() == b"".join(parts)
        assert body["video_id"] == compute_video_id(body["path"])
        # partial file cleaned up, registry entry gone
        assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]
        assert client.put(f"/api/upload/{up['upload_id']}/part/3", content=b"x").status_code
== 404


    def test_same_filename_twice_does_not_overwrite(env):
        client = env[0]
        paths = []
        for content in (b"first", b"second"):
            up = _init(client).json()
            client.put(f"/api/upload/{up['upload_id']}/part/0", content=content)
            done = client.post(f"/api/upload/{up['upload_id']}/complete",
json={"total_parts": 1})
            paths.append(done.json()["path"])
        assert paths[0] != paths[1]
        with open(paths[0], "rb") as fh:
            assert fh.read() == b"first"  # first upload untouched
        with open(paths[1], "rb") as fh:
            assert fh.read() == b"second"


    # --- sequencing & caps ------------------------------------------------------

    def test_out_of_order_part_is_409(env):
        client = env[0]
        up = _init(client).json()
        assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 409


    def test_unknown_upload_id_is_404(env):
        client = env[0]
        assert client.put("/api/upload/nope/part/0", content=b"x").status_code == 404
        assert client.post("/api/upload/nope/complete", json={"total_parts": 1}).status_code
== 404
        assert client.delete("/api/upload/nope").status_code == 404


    def test_complete_with_wrong_part_count_is_409(env):
        client = env[0]
        up = _init(client).json()
        client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
        assert client.post(
            f"/api/upload/{up['upload_id']}/complete", json={"total_parts": 2}).status_code
== 409
        # zero-part complete is also a 409
        up2 = _init(client).json()
        assert client.post(
            f"/api/upload/{up2['upload_id']}/complete", json={"total_parts": 0}).status_code
== 409


    def test_oversize_part_aborts_upload(env):
        client, db, tmp_path, cfg = env
        cfg.security.max_part_mb = 1
        up = _init(client).json()
        r = client.put(f"/api/upload/{up['upload_id']}/part/0",
                       content=b"x" * (1024 * 1024 + 10))
        assert r.status_code == 413
        # aborted: id unknown afterwards, no partials on disk
        assert client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x").status_code
```

```
== 404
139         assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]
140
141
142     def test_total_cap_across_parts_aborts_upload(env):
143         client, db, tmp_path, cfg = env
144         cfg.security.max_upload_mb = 1
145         up = _init(client).json()
146         ok = client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x" * (700 *
1024))
147         assert ok.status_code == 200
148         r = client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x" * (700 * 1024))
149         assert r.status_code == 413
150
151
152     def test_abort_endpoint_removes_state(env):
153         client, db, tmp_path, cfg = env
154         up = _init(client).json()
155         client.put(f"/api/upload/{up['upload_id']}/part/0", content=b"x")
156         assert client.delete(f"/api/upload/{up['upload_id']}").status_code == 200
157         assert client.put(f"/api/upload/{up['upload_id']}/part/1", content=b"x").status_code
== 404
158         assert not [f for f in os.listdir(cfg.security.upload_dir) if
f.startswith(".partial-")]
159
160
161     # --- download (B3)
----------------------------------------------------------------------
162
163     def test_download_unknown_video_404(env):
164         client = env[0]
165         assert client.get("/api/highlights/nope/reel.mp4").status_code == 404
166
167
168     def test_download_traversal_filename_rejected(env):
169         client, db, tmp_path, cfg = env
170         db.add_video("vid", "/x.mp4", "processed", [])
171         r = client.get("/api/highlights/vid/..%2Fevil.mp4")
172         assert r.status_code in (400, 404)  # 400 from safe-name; 404 if the router
normalizes
173         r2 = client.get("/api/highlights/vid/a%5Cb.mp4")  # backslash separator
174         assert r2.status_code == 400
175
176
177     def test_download_missing_file_404(env):
178         client, db, tmp_path, cfg = env
179         db.add_video("vid", "/x.mp4", "processed", [])
180         assert client.get("/api/highlights/vid/reel.mp4").status_code == 404
181
182
183     def test_download_streams_compiled_file(env):
184         client, db, tmp_path, cfg = env
185         db.add_video("vid", "/x.mp4", "processed", [])
186         blob = b"\x00\x00fake-mp4-bytes" * 100
187         with open(os.path.join(cfg.output.output_dir, "reel.mp4"), "wb") as fh:
188             fh.write(blob)
189         r = client.get("/api/highlights/vid/reel.mp4")
190         assert r.status_code == 200
191         assert r.content == blob
192         assert r.headers["content-type"].startswith("video/mp4")
193         assert "reel.mp4" in r.headers.get("content-disposition", "")
194
```

13. assistant (2026-06-18T08:05:56.000Z)

The critical understanding of the test seam:
- `monkeypatch.setattr(m, "global_config", cfg)` ? sets `global_config` on `backend.main`. The uploads module must read `backend.main.global_config` so this monkeypatch flows through.
- `monkeypatch.setattr(m, "_uploads", {})` ? replaces the `_uploads` dict on `backend.main`. After the move, `_uploads` lives on `backend.api.uploads`, so this monkeypatch target must be retargeted.

So the design: `uploads.py` references `backend.main.global_config` dynamically (via `from backend import main as main_module` then `main_module.global_config`), preserving the live-startup-config seam AND the `global_config` monkeypatch. The `_uploads` registry moves to `uploads.py`, and the test's `monkeypatch.setattr(m, "_uploads", {})` retargets to the uploads module.

Wait ? but there's a subtlety. The `_uploads_lock` and `_uploads` dict need to live in one place. If `_abort_upload`, `upload_init` etc. all live in `uploads.py` and reference the module-level `_uploads` there, then monkeypatching `uploads_api._uploads` must rebind the name the endpoints read. Since the endpoints reference the module global `_uploads` by name at call time (not captured), rebinding `uploads.uploads._uploads` works. Good.

Let me check the `backend/config` to understand `security` config, and check whether there's a circular import risk (main imports uploads at module level, uploads imports main).

14. user (2026-06-18T08:05:59.169Z)

```
annotations
api
assets
config
eval
features
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
tools
utils
---
```

15. assistant (2026-06-18T08:06:01.598Z)

The grep for config models returned nothing ? let me check the path and the backend/api dir contents (there's an `api` dir already with `static`).

16. user (2026-06-18T08:06:03.425Z)

```
total 4
drwxr-xr-x 1 avidu 197609 0 Jun 18 13:35 .
drwxr-xr-x 1 avidu 197609 0 Jun 18 13:35 ..
drwxr-xr-x 1 avidu 197609 0 Jun 18 13:35 static
---__init__?---
NO __init__.py
---config dir---
__init__.py
loader.py
models.py
```

17. assistant (2026-06-18T08:06:08.279Z)

`backend/api` has no `__init__.py` yet (only a `static/` subdir). I need to create `backend/api/__init__.py`. Let me confirm the security config fields and check how FastAPI route registration order matters (the upload endpoints currently use `@app.post`, etc. ? moving to an

`APIRouter` and `include_router` keeps the same paths). Let me also verify there are no other references to these symbols elsewhere in the codebase.

18. user (2026-06-18T08:06:09.901Z)

```
256:class SecurityConfig(BaseModel):
264:    # contain upload_dir or /api/process will reject the uploaded paths.
265:    upload_dir: str = "output/uploads"
268:    max_upload_mb: int = 4096
269:    max_part_mb: int = 95
270:    allowed_upload_extensions: List[str] = ["mp4", "mov"]
290:    security: SecurityConfig = Field(default_factory=SecurityConfig)
```

19. user (2026-06-18T08:06:11.308Z)

```
Found 3 files
tests\test_upload_download.py
docs\CODE_MAP.md
backend\main.py
```

20. assistant (2026-06-18T08:06:14.893Z)

Only `backend/main.py` and the test reference these symbols (and `docs/CODE_MAP.md` mentions them in prose ? but I'm restricted to editing only the named files: `backend/main.py`, new `backend/api/uploads.py`, `backend/api/__init__.py`, and the test). Let me check what CODE_MAP says to ensure I'm not breaking a documented contract, then build.

21. user (2026-06-18T08:06:16.103Z)

```
114-[Omitted long context line]
115:[Omitted long matching line]
116-  - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
```

22. user (2026-06-18T08:06:20.013Z)

```
113       - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
114       - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::_get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests
monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
115       - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST
/api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate ->
`{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY
sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach
-> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble
-> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs
/api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state
(`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits);
```

per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
116   - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.

23. assistant (2026-06-18T08:06:33.706Z)

I have full understanding. CODE_MAP is documentation only (not in my editable scope, and the symbols stay importable from `backend.main` via re-export if I want ? but the task says replace inline defs with the router include).

Now the key design decision for the config seam. The endpoints reference `global_config` via `_upload_cfg()` which does `getattr(global_config, "security", None)`. After moving, `uploads.py` must resolve `backend.main.global_config` at call time. Since `main.py` will import `uploads` at module load, and `uploads` would import `main` ? there's a potential circular import. To avoid it, `uploads.py` should do the `import backend.main` lazily inside `_upload_cfg()` (function body), OR use a module-level `from backend import main as main_module` placed carefully.

Circular import analysis: `main.py` will have `from backend.api import uploads as uploads_api` near where it sets up the app. At that point, importing `uploads` triggers `uploads`'s module body. If `uploads` does `from backend import main` at top level, `backend.main` is already partway-initialized (mid-import), so the name `main` would resolve to the partially-initialized module object ? that's fine as long as `uploads` doesn't access `main.global_config` at import time (it only accesses it inside `_upload_cfg()` at request time). So `import backend.main as _main` at module level in `uploads.py` is safe. But to be maximally safe against import-order issues, I'll import inside the function. Actually, the cleanest and most robust: `import backend.main` lazily inside `_upload_cfg` and `upload_complete`... but `upload_complete` only uses `_upload_cfg`'s result + `compute_video_id`. Only `_upload_cfg` touches `global_config`.

Let me reconsider: the test monkeypatches `m.global_config` (i.e., `backend.main.global_config`). So `uploads._upload_cfg` MUST read `backend.main.global_config` dynamically. I'll do a module-level `from backend import main as _main_module` ? but that creates the circular reference at the moment `main` imports `uploads`. Let me verify Python handles this: when `main` executes `from backend.api import uploads as uploads_api`, Python starts executing `uploads.py`. If `uploads.py` top-level has `from backend import main as _main_module`, Python sees `backend.main` is already in `sys.modules` (partially initialized) and binds `_main_module` to that partial module object ? no error, because it's a module-object binding, not an attribute access. Good. Then at request time, `_main_module.global_config` reads the live attribute. This works and respects monkeypatch.

I'll use the module-level `from backend import main as _main` for clarity, accessing `_main.global_config` at call time. Let me write `uploads.py`.

24. assistant (2026-06-18T08:06:56.112Z)

"""Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.

Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side under security.upload_dir, then submitted to /api/process by the returned path. Upload state is in-process by design (this is the single-box alpha): a server restart aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir arrives with PR-3 identity scoping.

The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``). ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always reads the live value the server set at startup (and so tests that monkeypatch ``backend.main.global_config`` flow through unchanged).
"""

```
import os
import threading
import uuid
from typing import Any, Dict, Optional

from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

import backend.main as _main  # read the live module-level global_config set at startup
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import safe_output_filename

router = APIRouter()

_uploads: Dict[str, Dict[str, Any]] = {}
_uploads_lock = threading.Lock()


class UploadInitRequest(BaseModel):
    filename: str
    total_size_bytes: Optional[int] = None


class UploadCompleteRequest(BaseModel):
    total_parts: int


def _upload_cfg():
    sec = getattr(_main.global_config, "security", None)
    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions", None) or
["mp4", "mov"])]
    return upload_dir, max_bytes, part_bytes, exts


def _abort_upload(upload_id: str) -> None:
    with _uploads_lock:
        st = _uploads.pop(upload_id, None)
    if st:
        try:
            os.remove(st["tmp_path"])
        except OSError:
            pass


@router.post("/api/upload/init")
def upload_init(req: UploadInitRequest):
    """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
    upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
    try:
        name = safe_output_filename(req.filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    ext = os.path.splitext(name)[1].lower().lstrip(".")
    if ext not in exts:
        raise HTTPException(status_code=400,
                            detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}")
    if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
        raise HTTPException(status_code=413,
                            detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload
limit.")
    os.makedirs(upload_dir, exist_ok=True)
    upload_id = uuid.uuid4().hex
```

```
    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
    open(tmp_path, "wb").close()
    with _uploads_lock:
        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes": 0}
    return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0"}


@router.put("/api/upload/{upload_id}/part/{index}")
async def upload_part(upload_id: str, index: int, request: Request):
    """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
    _, max_bytes, part_bytes, _ = _upload_cfg()
    with _uploads_lock:
        st = _uploads.get(upload_id)
    if st is None:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    if index != st["parts"]:
        raise HTTPException(status_code=409,
                            detail=f"Out-of-order part: expected {st['parts']}, got {index}
(parts are sequential)")

    received = 0
    overflow = None
    with open(st["tmp_path"], "ab") as fh:
        async for chunk in request.stream():
            received += len(chunk)
            if received > part_bytes:
                overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                break
            if st["bytes"] + received > max_bytes:
                overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                break
            fh.write(chunk)
    if overflow:
        _abort_upload(upload_id)
        raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")

    with _uploads_lock:
        st["parts"] += 1
        st["bytes"] += received
    return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


@router.post("/api/upload/{upload_id}/complete")
def upload_complete(upload_id: str, req: UploadCompleteRequest):
    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
    upload_dir, _, _, _ = _upload_cfg()
    with _uploads_lock:
        st = _uploads.get(upload_id)
    if st is None:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    if st["parts"] == 0 or st["parts"] != req.total_parts:
        raise HTTPException(status_code=409,
                            detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
    final_path = os.path.join(upload_dir, st["filename"])
    if os.path.exists(final_path):  # never overwrite an earlier upload with the same name
        stem, ext = os.path.splitext(st["filename"])
        final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
    os.replace(st["tmp_path"], final_path)
    with _uploads_lock:
        _uploads.pop(upload_id, None)
```

```
    video_id = compute_video_id(final_path)
    return {"path": os.path.abspath(final_path), "video_id": video_id,
            "size_bytes": os.path.getsize(final_path),
            "next": "POST /api/process with this path"}


@router.delete("/api/upload/{upload_id}")
def upload_abort(upload_id: str):
    """Abort an in-flight upload and delete its partial file."""
    with _uploads_lock:
        known = upload_id in _uploads
    if not known:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    _abort_upload(upload_id)
    return {"status": "aborted", "upload_id": upload_id}
```

25. user (2026-06-18T08:06:56.114Z)

```
<tool_use_error>This agent is isolated in the worktree C:\Users\avidu\Projects\badminton-
highlight-indexer\.claude\worktrees\wf_87c49086-be4-7. Edit the worktree copy of this file
instead of the shared-checkout path.</tool_use_error>
```

26. assistant (2026-06-18T08:06:59.678Z)

```
I need to write to the worktree path, not the shared checkout path.
```

27. assistant (2026-06-18T08:07:21.671Z)

```
"""Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.

Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
under security.upload_dir, then submitted to /api/process by the returned path.
Upload state is in-process by design (this is the single-box alpha): a server restart
aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
arrives with PR-3 identity scoping.

The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
reads the live value the server set at startup (and so tests that monkeypatch
``backend.main.global_config`` flow through unchanged).
"""
import os
import threading
import uuid
from typing import Any, Dict, Optional

from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

import backend.main as _main  # read the live module-level global_config set at startup
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import safe_output_filename

router = APIRouter()

_uploads: Dict[str, Dict[str, Any]] = {}
_uploads_lock = threading.Lock()


class UploadInitRequest(BaseModel):
    filename: str
    total_size_bytes: Optional[int] = None
```

```
class UploadCompleteRequest(BaseModel):
    total_parts: int


def _upload_cfg():
    sec = getattr(_main.global_config, "security", None)
    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
    exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions", None) or
["mp4", "mov"])]
    return upload_dir, max_bytes, part_bytes, exts


def _abort_upload(upload_id: str) -> None:
    with _uploads_lock:
        st = _uploads.pop(upload_id, None)
    if st:
        try:
            os.remove(st["tmp_path"])
        except OSError:
            pass


@router.post("/api/upload/init")
def upload_init(req: UploadInitRequest):
    """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
    upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
    try:
        name = safe_output_filename(req.filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    ext = os.path.splitext(name)[1].lower().lstrip(".")
    if ext not in exts:
        raise HTTPException(status_code=400,
                            detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}")
    if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
        raise HTTPException(status_code=413,
                            detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload
limit.")
    os.makedirs(upload_dir, exist_ok=True)
    upload_id = uuid.uuid4().hex
    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
    open(tmp_path, "wb").close()
    with _uploads_lock:
        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes": 0}
    return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0"}


@router.put("/api/upload/{upload_id}/part/{index}")
async def upload_part(upload_id: str, index: int, request: Request):
    """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
    _, max_bytes, part_bytes, _ = _upload_cfg()
    with _uploads_lock:
        st = _uploads.get(upload_id)
    if st is None:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    if index != st["parts"]:
        raise HTTPException(status_code=409,
                            detail=f"Out-of-order part: expected {st['parts']}, got {index}
(parts are sequential)")

    received = 0
    overflow = None
```

```
    with open(st["tmp_path"], "ab") as fh:
        async for chunk in request.stream():
            received += len(chunk)
            if received > part_bytes:
                overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                break
            if st["bytes"] + received > max_bytes:
                overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                break
            fh.write(chunk)
    if overflow:
        _abort_upload(upload_id)
        raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")

    with _uploads_lock:
        st["parts"] += 1
        st["bytes"] += received
    return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


@router.post("/api/upload/{upload_id}/complete")
def upload_complete(upload_id: str, req: UploadCompleteRequest):
    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
    upload_dir, _, _, _ = _upload_cfg()
    with _uploads_lock:
        st = _uploads.get(upload_id)
    if st is None:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    if st["parts"] == 0 or st["parts"] != req.total_parts:
        raise HTTPException(status_code=409,
                            detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
    final_path = os.path.join(upload_dir, st["filename"])
    if os.path.exists(final_path):  # never overwrite an earlier upload with the same name
        stem, ext = os.path.splitext(st["filename"])
        final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
    os.replace(st["tmp_path"], final_path)
    with _uploads_lock:
        _uploads.pop(upload_id, None)
    video_id = compute_video_id(final_path)
    return {"path": os.path.abspath(final_path), "video_id": video_id,
            "size_bytes": os.path.getsize(final_path),
            "next": "POST /api/process with this path"}


@router.delete("/api/upload/{upload_id}")
def upload_abort(upload_id: str):
    """Abort an in-flight upload and delete its partial file."""
    with _uploads_lock:
        known = upload_id in _uploads
    if not known:
        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
    _abort_upload(upload_id)
    return {"status": "aborted", "upload_id": upload_id}
```

28. user (2026-06-18T08:07:21.710Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\backend\api\uploads.py (file state is current in
your context ? no need to Read it back)
```

29. assistant (2026-06-18T08:07:24.653Z)

Now create the `backend/api/__init__.py`. Let me check whether `backend/utils/paths` exports `safe_output_filename` (it's imported in main.py line 33, so yes). Now create the package init.

30. assistant (2026-06-18T08:07:25.732Z)

```
"""HTTP API surface for the indexer (FastAPI routers)."""
```

31. user (2026-06-18T08:07:25.765Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-7\backend\api__init__.py (file state is current in your context ? no need to Read it back)

32. assistant (2026-06-18T08:07:31.861Z)

Now there's a circular-import ordering concern I must resolve carefully. `uploads.py` does `import backend.main as _main` at top level. When `main.py` executes `from backend.api import uploads as uploads_api`, this happens partway through `main.py`'s module execution. At that point `backend.main` IS in `sys.modules` (partially initialized), so `import backend.main as _main` in uploads.py binds successfully without re-executing main.

But the placement matters: `main.py` must call `from backend.api import uploads` and `app.include_router(...)` at a point where `app` already exists. `app` is created at line 38. So I'll place the include_router right where the inline upload defs currently are (after line 400), or right after the CORS/static setup. I'll replace the inline upload block (lines 401-532) with the import + include_router. That keeps it in the same location.

Now let me edit `main.py`. First, remove the now-unused imports check: `threading` and `uuid` are used elsewhere in main.py? Let me verify before removing anything ? I must NOT remove imports still in use.

33. user (2026-06-18T08:07:33.058Z)

```
5:import threading
7:import uuid
21:from fastapi import FastAPI, HTTPException, Request
23:from pydantic import BaseModel
33:from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
87:class ProcessRequest(BaseModel):
93:class QueryRequest(BaseModel):
100:class CompileRequest(BaseModel):
336:    timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
369:    job_id = uuid.uuid4().hex
410:_uploads_lock = threading.Lock()
413:class UploadInitRequest(BaseModel):
418:class UploadCompleteRequest(BaseModel):
446:        name = safe_output_filename(req.filename)
457:    upload_id = uuid.uuid4().hex
467:async def upload_part(upload_id: str, index: int, request: Request):
543:        name = safe_output_filename(filename)
588:        out_name = safe_output_filename(req.output_filename)
```

34. assistant (2026-06-18T08:07:41.582Z)

Important findings for which imports stay in main.py after removing the upload block:
- `threading` ? still used at line 336. KEEP.
- `uuid` ? still used at line 369. KEEP.
- `Request` (from fastapi) ? only used at line 467 (upload_part), which is being removed. Let me verify there's no other use. The grep shows `Request` only at line 21 (import) and 467. So `Request` becomes unused in main.py. But `HTTPException` and `FastAPI` are still used. I must remove `Request` from the fastapi import to keep ruff clean (unused import).
- `safe_output_filename` ? still used at lines 543, 588. KEEP (and `resolve_under_root`, `validate_input_path`).
- `BaseModel` ? still used at lines 87, 93, 100. KEEP.

Let me double-check `Request` truly has no other usage in main.py.

35. user (2026-06-18T08:07:42.680Z)

```
21:from fastapi import FastAPI, HTTPException, Request
87:class ProcessRequest(BaseModel):
93:class QueryRequest(BaseModel):
100:class CompileRequest(BaseModel):
123:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
256:def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
257:    """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
259:    the original submit. The row stores exactly the ProcessRequest fields."""
260:    return ProcessRequest(
347:def process_video(req: ProcessRequest):
413:class UploadInitRequest(BaseModel):
418:class UploadCompleteRequest(BaseModel):
442:def upload_init(req: UploadInitRequest):
467:async def upload_part(upload_id: str, index: int, request: Request):
501:def upload_complete(upload_id: str, req: UploadCompleteRequest):
562:def query_play_segments(req: QueryRequest):
573:def compile_highlights(req: CompileRequest):
```

36. assistant (2026-06-18T08:07:53.926Z)

Confirmed: the only use of `Request` (capital-R, the FastAPI class) is the import on line 21 and the `upload_part` signature on line 467 (which is being removed). The other matches are `ProcessRequest`/`QueryRequest`/`CompileRequest` (different names). So I'll drop `Request` from the fastapi import line.

Now let me make the edits to main.py. First, replace the upload block (lines 401-532) with the router include.

37. user (2026-06-18T08:08:18.566Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

38. assistant (2026-06-18T08:08:22.559Z)

The Read earlier was against the shared-checkout path. I need to read the worktree copy first.

39. user (2026-06-18T08:08:24.150Z)

```
399         }
400
401     # --- Chunked upload (B2, PR-2) ----------------------------------------------------
402     # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403     # proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
404     # under security.upload_dir, then submitted to /api/process by the returned path.
405     # Upload state is in-process by design (this is the single-box alpha): a server restart
406     # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407     # arrives with PR-3 identity scoping.
408
409     _uploads: Dict[str, Dict[str, Any]] = {}
410     _uploads_lock = threading.Lock()
411
412
413     class UploadInitRequest(BaseModel):
414         filename: str
415         total_size_bytes: Optional[int] = None
416
417
418     class UploadCompleteRequest(BaseModel):
419         total_parts: int
420
```

```

    def _upload_cfg():
        sec = getattr(global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @app.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(status_code=400,
                                detail=f"Extension '.{ext}' not allowed. Allowed:
{sorted(exts)}")
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(status_code=413,
                                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
        os.makedirs(upload_dir, exist_ok=True)
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
        return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
                "next_part": f"/api/upload/{upload_id}/part/0"}


    @app.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(status_code=409,
                                detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")

        received = 0
        overflow = None
        with open(st["tmp_path"], "ab") as fh:
```

```
481             async for chunk in request.stream():
482                 received += len(chunk)
483                 if received > part_bytes:
484                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                     break
486                 if st["bytes"] + received > max_bytes:
487                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                     break
489                 fh.write(chunk)
490         if overflow:
491             _abort_upload(upload_id)
492             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493 
494         with _uploads_lock:
495             st["parts"] += 1
496             st["bytes"] += received
497         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498 
499 
500     @app.post("/api/upload/{upload_id}/complete")
501     def upload_complete(upload_id: str, req: UploadCompleteRequest):
502         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503         upload_dir, _, _, _ = _upload_cfg()
504         with _uploads_lock:
505             st = _uploads.get(upload_id)
506         if st is None:
507             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
508         if st["parts"] == 0 or st["parts"] != req.total_parts:
509             raise HTTPException(status_code=409,
510                                 detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511         final_path = os.path.join(upload_dir, st["filename"])
512         if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
513             stem, ext = os.path.splitext(st["filename"])
514             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515         os.replace(st["tmp_path"], final_path)
516         with _uploads_lock:
517             _uploads.pop(upload_id, None)
518         video_id = compute_video_id(final_path)
519         return {"path": os.path.abspath(final_path), "video_id": video_id,
520                 "size_bytes": os.path.getsize(final_path),
521                 "next": "POST /api/process with this path"}
522 
523 
524     @app.delete("/api/upload/{upload_id}")
525     def upload_abort(upload_id: str):
526         """Abort an in-flight upload and delete its partial file."""
527         with _uploads_lock:
528             known = upload_id in _uploads
529         if not known:
530             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531         _abort_upload(upload_id)
532         return {"status": "aborted", "upload_id": upload_id}
533 
534 
```

40. assistant (2026-06-18T08:08:26.360Z)

Now I'll make the replacement edit on the worktree copy.

41. user (2026-06-18T08:08:49.495Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-7\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

42. assistant (2026-06-18T08:08:51.534Z)

Now remove the unused `Request` import from the fastapi line in main.py.

43. user (2026-06-18T08:08:53.424Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-7\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. assistant (2026-06-18T08:08:57.424Z)

Now retarget the test monkeypatch for `_uploads`. The test sets `monkeypatch.setattr(m, "_uploads", {})`. Since `_uploads` now lives on `backend.api.uploads`, I retarget that one line to the new module. The `global_config` and `db_instance` monkeypatches stay on `m` (backend.main) because uploads reads `_main.global_config`.

Let me edit the test file.

45. user (2026-06-18T08:08:59.633Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

46. assistant (2026-06-18T08:09:02.117Z)

Need to read the worktree copy of the test first.

47. user (2026-06-18T08:09:03.060Z)

```
16
17      import backend.main as m
18      from backend.infrastructure.database import Database
19      from backend.config.models import AppConfig
20      from backend.utils.hashing import compute_video_id
21
22
23      @pytest.fixture
24      def env(tmp_path, monkeypatch):
25          db = Database(str(tmp_path / "t.db"))
26          cfg = AppConfig()
27          cfg.output.output_dir = str(tmp_path / "out")
28          cfg.security.upload_dir = str(tmp_path / "up")
29          os.makedirs(cfg.output.output_dir, exist_ok=True)
30          monkeypatch.setattr(m, "db_instance", db)
31          monkeypatch.setattr(m, "global_config", cfg)
32          monkeypatch.setattr(m, "_uploads", {})  # isolate upload state per test
33          client = TestClient(m.app)
34          return client, db, tmp_path, cfg
35
```

48. user (2026-06-18T08:09:07.272Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-7\tests\test_upload_download.py has been updated successfully. (file state is current in your context ? no need to Read it back)

49. user (2026-06-18T08:09:12.101Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\tests\test_upload_download.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

50. assistant (2026-06-18T08:09:15.503Z)

Now let me verify there's no remaining reference to `Dict`/`Any`/`Optional` that I might have orphaned in main.py ? those typing imports are still used elsewhere (line 24 imports them; `_uploads: Dict[str, Dict[str, Any]]` is removed but plenty of other uses remain). And check the circular import actually works by doing a quick import test before running the full suite.

51. user (2026-06-18T08:09:29.547Z)

```
main imports OK
routes: ['/api/upload/init', '/api/upload/{upload_id}', '/api/upload/{upload_id}/complete',
'/api/upload/{upload_id}/part/{index}']
app has upload routes: ['/api/upload/init', '/api/upload/{upload_id}',
'/api/upload/{upload_id}/complete', '/api/upload/{upload_id}/part/{index}']
```

52. assistant (2026-06-18T08:09:34.232Z)

Imports resolve, no circular-import failure, and all 4 upload routes are registered on the app via the router. Now let me verify the import-order robustness: what if `backend.api.uploads` is imported FIRST (before `backend.main` finishes)? Since `uploads` does `import backend.main as _main` at top level, and if a test imports `uploads` first, Python will fully import `backend.main` (which then imports `uploads` again ? but `uploads` is mid-init in sys.modules). Let me test that order too, since `conftest` or test collection order could trigger it.

53. user (2026-06-18T08:09:50.177Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from backend.api import uploads as u; print('uploads-first import OK'); print('global_config
seam:', u._main.global_config); import backend.main as m; print('same module object:', u._main
is m)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\backend\api\uploads.py", line 23, in <module>
    import backend.main as _main  # read the live module-level global_config set at startup
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\backend\main.py", line 408, in <module>
    app.include_router(uploads_api.router)
                       ^^^^^^^^^^^^^^^^^^
AttributeError: partially initialized module 'backend.api.uploads' from
'C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\backend\api\uploads.py' has no attribute 'router'
(most likely due to a circular import)
```

54. assistant (2026-06-18T08:09:55.451Z)

This is the circular-import trap I needed to catch. When `uploads` is imported first, it hits `import backend.main` before defining `router`; `main` then tries `uploads_api.router` which doesn't exist yet.

The fix: in `uploads.py`, don't import `backend.main` at module top level. Instead resolve it lazily inside `_upload_cfg()` (the only place that needs `global_config`). This breaks the cycle: importing `uploads` no longer triggers importing `main` at module-load time. The `import backend.main` happens at request time when both modules are fully initialized. This preserves the live-config seam (it reads `backend.main.global_config` dynamically) and respects monkeypatch.

Let me update `uploads.py`.

55. user (2026-06-18T08:09:58.513Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\backend\api\uploads.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

56. user (2026-06-18T08:10:03.457Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_87c49086-be4-7\backend\api\uploads.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

57. assistant (2026-06-18T08:10:06.564Z)

Now re-test both import orders.

58. user (2026-06-18T08:10:25.551Z)

```
=== order A: main first ===
OK
['/api/upload/init', '/api/upload/{upload_id}', '/api/upload/{upload_id}/complete',
'/api/upload/{upload_id}/part/{index}']
=== order B: uploads first ===
OK
cfg seam reads main: ('backend.main', 'main', 'getattr')
```

59. assistant (2026-06-18T08:10:27.907Z)

Both import orders work now. Let me run the targeted upload test first, then the full gate.

60. user (2026-06-18T08:10:39.232Z)

```
...............                                                          [100%]
15 passed in 7.53s
```

61. assistant (2026-06-18T08:10:42.166Z)

All 15 upload/download tests pass. Now run the full gate: the suite, ruff, mypy, then coverage.

62. user (2026-06-18T08:12:11.682Z)

```
======================================================================
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
======================================================================
.....................ss.............................................. [  8%]
........................................................................ [ 16%]
........................................................................ [ 25%]
........................................................................ [ 33%]
........................................................................ [ 41%]
........................................................................ [ 50%]
........................................................................ [ 58%]
........................................................................ [ 66%]
........................................................................ [ 75%]
.......sssss............................................................ [ 83%]
........................................................................ [ 91%]
.........................................................................  [100%]
856 passed, 7 skipped in 85.21s (0:01:25)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

63. assistant (2026-06-18T08:12:15.749Z)

Full suite green: 856 passed, 7 skipped. Now ruff and mypy.

64. user (2026-06-18T08:13:10.341Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 106 source files
```

65. assistant (2026-06-18T08:13:13.001Z)

Ruff clean, mypy success (notes only, no errors ? and these annotation-unchecked notes are pre-existing, not from my files). Now the coverage gate.

66. user (2026-06-18T08:14:55.297Z)

```
backend\tools\__init__.py                              0      0   100%
backend\tools\export.py                               75      2    97%
backend\tools\rally_slicer.py                        104     47    55%
backend\utils\__init__.py                              0      0   100%
backend\utils\geometry.py                             29      0   100%
backend\utils\hashing.py                               9      0   100%
backend\utils\logging_setup.py                        11      0   100%
backend\utils\paths.py                                27      3    89%
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\video.py                                25      3    88%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             108     11    90%
----------------------------------------------------------------------
TOTAL                                               8021   1544    81%
Required test coverage of 70% reached. Total coverage: 80.75%
856 passed, 7 skipped, 7 warnings in 96.97s (0:01:36)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

67. assistant (2026-06-18T08:14:58.881Z)

All four gates green: suite 856 passed/7 skipped, ruff clean, mypy success, coverage 80.75% (well above the 70 floor). Let me check coverage of the new uploads.py module specifically and the main.py, then verify the diff.

68. user (2026-06-18T08:15:00.407Z)

```
 M backend/main.py
 M tests/test_upload_download.py
?? backend/api/__init__.py
?? backend/api/uploads.py
=== DIFFSTAT ===
 backend/main.py               | 138 ++++----------------------------------------
 tests/test_upload_download.py |   3 +-
 2 files changed, 9 insertions(+), 132 deletions(-)
```

69. assistant (2026-06-18T08:15:03.780Z)

Note the diffstat doesn't include the new untracked files yet (git diff --stat only shows tracked). Let me get the full picture by staging the explicit paths and reviewing the staged diffstat. First, let me re-verify HEAD/branch safety per the concurrent-session protocol before committing.

70. user (2026-06-18T08:15:05.594Z)

```
branch:
refactor/main-uploads-apirouter
HEAD:
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
=== commits not on origin/master (should be NONE before commit) ===
```

71. assistant (2026-06-18T08:15:07.879Z)

Clean: branch is correct, HEAD == origin/master, no sibling commits. Now stage the explicit paths and review the full staged diffstat.

72. user (2026-06-18T08:15:09.765Z)

```
warning: in the working copy of 'backend/api/__init__.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'backend/api/uploads.py', LF will be replaced by CRLF the next
time Git touches it
=== staged diffstat ===
 backend/api/__init__.py       |   1 +
 backend/api/uploads.py        | 155 ++++++++++++++++++++++++++++++++++++++++++++
 backend/main.py               | 138 ++++----------------------------------
 tests/test_upload_download.py |   3 +-
 4 files changed, 165 insertions(+), 132 deletions(-)
=== staged status ===
A  backend/api/__init__.py
A  backend/api/uploads.py
M  backend/main.py
M  tests/test_upload_download.py
```

73. assistant (2026-06-18T08:15:12.347Z)

All four intended files staged, nothing else. Let me do a final review of the staged diff for main.py to confirm it's a clean cut (the move and the single import-line change), then commit.

74. user (2026-06-18T08:15:13.510Z)

```
diff --git a/backend/main.py b/backend/main.py
index 0f48c7e..40fe77e 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -18,7 +18,7 @@ logging.basicConfig(
     datefmt='%H:%M:%S'
 )
 logger = logging.getLogger(__name__)
-from fastapi import FastAPI, HTTPException, Request
+from fastapi import FastAPI, HTTPException
 from fastapi.middleware.cors import CORSMiddleware
 from pydantic import BaseModel
 from typing import List, Dict, Any, Optional
@@ -399,137 +399,13 @@ def get_job(job_id: str):
     }

 # --- Chunked upload (B2, PR-2) ----------------------------------------------------
-# Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
-# proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
```

```
-# under security.upload_dir, then submitted to /api/process by the returned path.
-# Upload state is in-process by design (this is the single-box alpha): a server restart
-# aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
-# arrives with PR-3 identity scoping.
+# The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives in
+# backend.api.uploads as a self-contained APIRouter. It resolves the live startup config via
+# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
+# sequential-part logic, and abort cleanup ? is unchanged.
+from backend.api import uploads as uploads_api

-_uploads: Dict[str, Dict[str, Any]] = {}
-_uploads_lock = threading.Lock()
-
-
-class UploadInitRequest(BaseModel):
-    filename: str
-    total_size_bytes: Optional[int] = None
-
-
-class UploadCompleteRequest(BaseModel):
-    total_parts: int
-
-
-def _upload_cfg():
-    sec = getattr(global_config, "security", None)
-    upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
-    max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
-    part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
-    exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions", None) or
["mp4", "mov"])]
-    return upload_dir, max_bytes, part_bytes, exts
-
-
-def _abort_upload(upload_id: str) -> None:
-    with _uploads_lock:
-        st = _uploads.pop(upload_id, None)
-    if st:
-        try:
-            os.remove(st["tmp_path"])
-        except OSError:
-            pass
-
-
-@app.post("/api/upload/init")
-def upload_init(req: UploadInitRequest):
-    """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
-    upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
-    try:
-        name = safe_output_filename(req.filename)
-    except ValueError as e:
-        raise HTTPException(status_code=400, detail=str(e)) from e
-    ext = os.path.splitext(name)[1].lower().lstrip(".")
-    if ext not in exts:
-        raise HTTPException(status_code=400,
-                            detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}")
-    if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
-        raise HTTPException(status_code=413,
-                            detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload
limit.")
-    os.makedirs(upload_dir, exist_ok=True)
-    upload_id = uuid.uuid4().hex
-    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
-    open(tmp_path, "wb").close()
-    with _uploads_lock:
-        _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0, "bytes": 0}
```

```
-    return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
-            "next_part": f"/api/upload/{upload_id}/part/0"}
-
-
-@app.put("/api/upload/{upload_id}/part/{index}")
-async def upload_part(upload_id: str, index: int, request: Request):
-    """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
-    _, max_bytes, part_bytes, _ = _upload_cfg()
-    with _uploads_lock:
-        st = _uploads.get(upload_id)
-    if st is None:
-        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
-    if index != st["parts"]:
-        raise HTTPException(status_code=409,
-                            detail=f"Out-of-order part: expected {st['parts']}, got {index}
(parts are sequential)")
-
-    received = 0
-    overflow = None
-    with open(st["tmp_path"], "ab") as fh:
-        async for chunk in request.stream():
-            received += len(chunk)
-            if received > part_bytes:
-                overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
-                break
-            if st["bytes"] + received > max_bytes:
-                overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
-                break
-            fh.write(chunk)
-    if overflow:
-        _abort_upload(upload_id)
-        raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")
-
-    with _uploads_lock:
-        st["parts"] += 1
-        st["bytes"] += received
-    return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
-
-
-@app.post("/api/upload/{upload_id}/complete")
-def upload_complete(upload_id: str, req: UploadCompleteRequest):
-    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
-    upload_dir, _, _, _ = _upload_cfg()
-    with _uploads_lock:
-        st = _uploads.get(upload_id)
-    if st is None:
-        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
-    if st["parts"] == 0 or st["parts"] != req.total_parts:
-        raise HTTPException(status_code=409,
-                            detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
-    final_path = os.path.join(upload_dir, st["filename"])
-    if os.path.exists(final_path):  # never overwrite an earlier upload with the same name
-        stem, ext = os.path.splitext(st["filename"])
-        final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
-    os.replace(st["tmp_path"], final_path)
-    with _uploads_lock:
-        _uploads.pop(upload_id, None)
-    video_id = compute_video_id(final_path)
-    return {"path": os.path.abspath(final_path), "video_id": video_id,
-            "size_bytes": os.path.getsize(final_path),
-            "next": "POST /api/process with this path"}
```

```
-
-
-@app.delete("/api/upload/{upload_id}")
-def upload_abort(upload_id: str):
-    """Abort an in-flight upload and delete its partial file."""
-    with _uploads_lock:
-        known = upload_id in _uploads
-    if not known:
-        raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
-    _abort_upload(upload_id)
-    return {"status": "aborted", "upload_id": upload_id}
+app.include_router(uploads_api.router)


 # --- Highlight download (B3, PR-2) ----------------------------------------------------
diff --git a/tests/test_upload_download.py b/tests/test_upload_download.py
index a06052d..3fba448 100644
--- a/tests/test_upload_download.py
+++ b/tests/test_upload_download.py
@@ -15,6 +15,7 @@ pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where
absen
 from fastapi.testclient import TestClient

 import backend.main as m
+from backend.api import uploads as uploads_api
 from backend.infrastructure.database import Database
 from backend.config.models import AppConfig
 from backend.utils.hashing import compute_video_id
@@ -29,7 +30,7 @@ def env(tmp_path, monkeypatch):
     os.makedirs(cfg.output.output_dir, exist_ok=True)
     monkeypatch.setattr(m, "db_instance", db)
     monkeypatch.setattr(m, "global_config", cfg)
-    monkeypatch.setattr(m, "_uploads", {})  # isolate upload state per test
+    monkeypatch.setattr(uploads_api, "_uploads", {})  # isolate upload state per test (registry
moved to backend.api.uploads)
     client = TestClient(m.app)
     return client, db, tmp_path, cfg
```

75. assistant (2026-06-18T08:15:17.085Z)

The diff is a clean cut-paste: the upload block is removed verbatim from main.py and replaced with the import + `include_router`, the unused `Request` import is dropped, and the test retargets only the `_uploads` monkeypatch. The endpoint bodies in uploads.py are byte-identical to the originals (only `@app.` ? `@router.` and the lazy `import backend.main as _main` in `_upload_cfg`).

Now commit with explicit paths and push.

76. user (2026-06-18T08:15:24.705Z)

```
[refactor/main-uploads-apirouter 9c61554] refactor(api): extract chunked-upload island from main
into backend.api.uploads router
 4 files changed, 165 insertions(+), 132 deletions(-)
 create mode 100644 backend/api/__init__.py
 create mode 100644 backend/api/uploads.py
=== committed ===
9c61554 refactor(api): extract chunked-upload island from main into backend.api.uploads router
```

77. assistant (2026-06-18T08:15:27.034Z)

Committed. Verify only my commit sits on top of origin/master, then push and open the PR.

78. user (2026-06-18T08:15:30.905Z)

```
=== commits on top of origin/master (must be ONLY mine) ===
9c61554 refactor(api): extract chunked-upload island from main into backend.api.uploads router
=== push ===
remote:
remote: Create a pull request for 'refactor/main-uploads-apirouter' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/main-
uploads-apirouter
remote:
branch 'refactor/main-uploads-apirouter' set up to track 'origin/refactor/main-uploads-
apirouter'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/main-uploads-apirouter -> refactor/main-uploads-apirouter
```

79. assistant (2026-06-18T08:15:36.891Z)

Pushed cleanly with only my commit. Now open the PR for owner review (no merge).

80. user (2026-06-18T08:15:54.138Z)

https://github.com/avidullu/badminton-highlight-indexer/pull/228

81. assistant (2026-06-18T08:15:58.746Z)

PR opened: https://github.com/avidullu/badminton-highlight-indexer/pull/228 (no merge). All four gates green. Let me return the structured result.

82. user (2026-06-18T08:16:10.060Z)

Structured output provided successfully

83. assistant (2026-06-18T08:16:19.149Z)

Done. The refactor is complete and the PR is open for owner review (not merged).

Summary of what landed on branch `refactor/main-uploads-apirouter` (PR #228):
- Created `backend/api/__init__.py` and `backend/api/uploads.py` ? the new module carries `router = APIRouter()` with the 4 upload routes, the `_uploads`/`_uploads_lock` registry, `_upload_cfg`/`_abort_upload` helpers, and the `UploadInitRequest`/`UploadCompleteRequest` models, all moved verbatim.
- `backend/main.py` replaced the ~130-line inline island with `from backend.api import uploads as uploads_api` + `app.include_router(uploads_api.router)`, and dropped the now-unused `Request` import.
- `tests/test_upload_download.py` retargeted only the `_uploads` monkeypatch to the new module.

The load-bearing detail: `_upload_cfg` resolves `backend.main.global_config` lazily (`import backend.main as _main` inside the function) ? this both preserves the live-startup-config seam and dodges a real circular-import failure I caught when `uploads` is imported before `main`. All four gates green (856 passed/7 skipped, ruff clean, mypy clean, coverage 80.75% vs the 70 floor).

One follow-up worth noting for the owner: `docs/CODE_MAP.md` still documents `::upload_init` and friends as living in `backend.main` ? it was outside this task's editing scope, so the map could use a touch-up to point at `backend.api.uploads`.